



DESARROLLO DE APLICACIONES ANDROID

Programación multimedia y dispositivos móviles. (PGL)

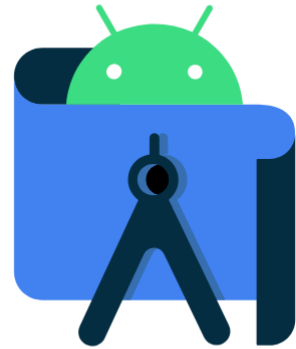




- Sistema operativo Android.
- Conceptos previos al desarrollo.
- Desarrollo de aplicaciones en Kotlin con Android Studio.

- Sistema operativo Android.
- Conceptos previos al desarrollo.
- Desarrollo de aplicaciones en Kotlin con Android Studio.

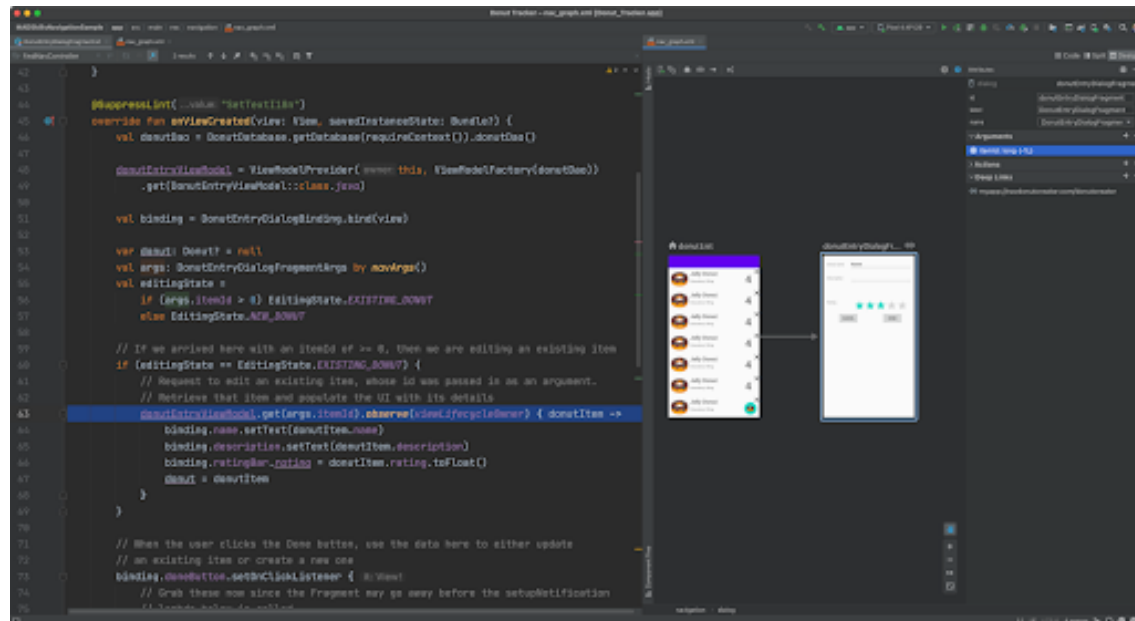
android studio



INTRODUCCIÓN

.....

- Android Studio es el IDE que vamos a utilizar para desarrollar aplicaciones para Android. Su apariencia es similar a IntelliJ IDEA (está basado en él).
- La guía de ayuda para realizar aplicaciones para Android se puede consultar en la siguiente página: <https://developer.android.com/guide/>
- Las aplicaciones de Android se escriben en **Kotlin**, lenguaje oficial de Android desde **2017** (anteriormente se realizaban en Java). Las herramientas de Android SDK compilan el código, que junto con los archivos de recursos y datos, se convierten en un archivo APK (sufijo de archivo .apk)



S.O. ANDROID



- El sistema operativo Android es un sistema Linux multiusuario en el que cada aplicación es un usuario diferente.
- El sistema le asigna a cada aplicación una ID de usuario de Linux única (solo el sistema utiliza la ID y la aplicación la desconoce).
- El sistema establece permisos para todos los archivos en una aplicación de modo que solo el ID de usuario asignado a esa aplicación pueda acceder a ellos.
- Cada proceso tiene su propio equipo virtual (EV), por lo que el código de una aplicación se ejecuta de forma independiente de otras aplicaciones.
- Cada aplicación tiene acceso solo a los componentes que necesita para llevar a cabo su trabajo. Esto crea un entorno seguro en el que una aplicación no puede acceder a partes del sistema para las que no tiene permiso. Aunque existen maneras de conceder permisos para que una aplicación pueda acceder a los datos de otra.
- Android permite crear hilos para ejecutar tareas de larga duración. Una vez terminada la tarea podemos regresar al hilo principal para actualizar la interfaz de usuario.

S.O. ANDROID

Componentes de una aplicación Android.

Los componentes en este contexto son bloques de creación, puntos de entrada a la aplicación y son:

Actividades

- Pantalla con interfaz de usuario
- Ejemplo: actividad que toma foto.
- Se implementa como subclase de *Activity*

Servicios

- Se ejecuta en segundo plano para procesos largos o remotos
- Ejemplo: reproducir música mientras usuario continúa en la aplicación
- Se implementa como subclase de *Service*

Proveedores de contenido

- Administra un conjunto compartido de datos de la app
- Se pueden almacenar datos en sistema archivos, SQLite, web, etc...
- Se implementa como una subclase de *ContentProvider*

Receptores de mensajes

- Responde a mensajes de todo el sistema
- Ejemplo: mensaje que indica que queda poca batería
- Se implementa como una subclase de *BroadcastReceiver* y cada receptor es un objeto *Intent*

CONCEPTOS

- MANIFIESTO
- Todas las aplicaciones deben tener un archivo AndroidManifest.xml en el directorio raíz. Éste proporciona información importante sobre la aplicación al sistema Android. Vemos un ejemplo de la información que contiene:



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.jacintorm.holadam">
4
5     <application
6         android:allowBackup="true"
7         android:icon="@mipmap/ic_launcher"
8         android:label="HolaDAM"
9         android:roundIcon="@mipmap/ic_launcher_round"
10        android:supportsRtl="true"
11        android:theme="@style/Theme.HolaDAM">
12        <activity
13            android:name=".MainActivity"
14            android:exported="true">
15            <intent-filter>
16                <action android:name="android.intent.action.MAIN" />
17
18                <category android:name="android.intent.category.LAUNCHER" />
19            </intent-filter>
20        </activity>
21    </application>
22
23 </manifest>
```


CONCEPTOS

- Los elementos que pueden aparecer en el fichero de manifiesto son los siguientes. Son los permitidos y no se podrán agregar otros. Obligatoriamente DEBEN aparecer y sólo una vez los de <manifest> y <application>. Para el resto de elementos pueden aparecer varias veces o no aparecer.

<action>

<activity> Declara una subclase Activity que implementa la interfaz de usuario.

<activity-alias>

<application> Contiene subelementos para declarar los componentes y sus valores (por defecto... etc...)

<category>

<data>

<grant-uri-permission>

<instrumentation>

CONCEPTOS

<intent-filter>

<manifest> El elemento raíz del fichero AndroidManifest.xml file. Debe contener un elemento <application>

<meta-data>

<permission>

<permission-group>

<permission-tree>

<provider>

<receiver>

<service>

<supports-screens>

<uses-configuration>

<uses-feature>

<uses-library>

<uses-permission>

<uses-sdk> Es un entero (nivel de API) que expresa la compatibilidad de una aplicación con una o más versiones de la plataforma de Android

CONCEPTOS

ACTIVITY

- Una Activity es un componente de la aplicación que contiene una pantalla con la que los usuarios pueden interactuar. La interfaz de usuario de las aplicaciones. Una aplicación puede tener muchas actividades y una de ellas será la principal, que es la que se muestra cuando se inicia la aplicación la primera vez. En cada activity podemos diferenciar dos partes:
 - La parte lógica sería la parte programática, que será una clase Kotlin (o Java) encargada de implementar las funcionalidades de la aplicación.
 - La parte gráfica consiste en un archivo xml, situado en la carpeta nombreproyecto/res/layout/archivo.xml, dónde se definen los componentes gráficos que formarán la interfaz gráfica del usuario y herramientas que les afectan.
 - Con el nuevo Kit de herramientas Jetpack Compose, no editarás ningún diseño XML ni usarás el editor de diseño. En su lugar, llamarás a las funciones de Jetpack Compose para decir qué elementos deseas, y el compilador de Compose hará el resto.

CONCEPTOS

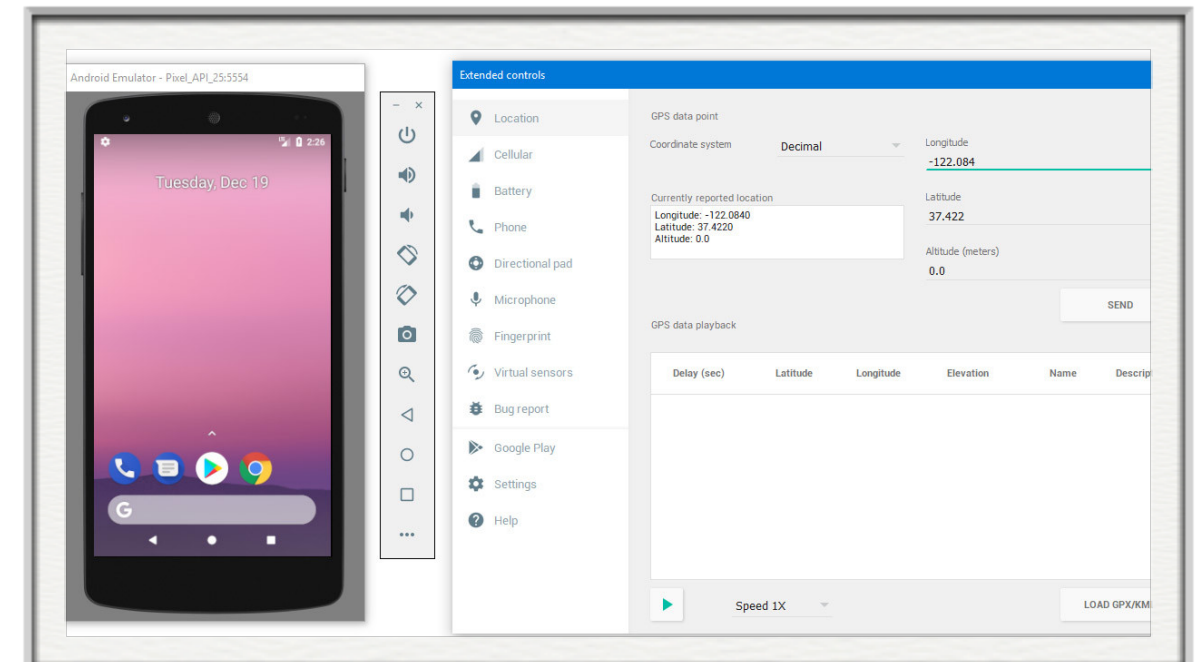
¿Cómo implementar una interfaz de usuario?

- La interfaz de usuario para una actividad se proporciona mediante una jerarquía de vistas (objetos derivados de la clase View). Cada vista controla un espacio rectangular específico dentro de la ventana de la actividad y pueden responder a la interacción del usuario. Por ejemplo, una vista podría ser un botón que inicie una acción cuando el usuario lo toca.
- La mayoría de las apps van a contener muchas vistas, es decir múltiples actividades. Una de ellas será la main activity, que será la primera en mostrarse. Luego, cada actividad podrá iniciar otra que realice diferentes acciones. Para usar las actividades en tu app, deben estar registradas en el manifest y manejar el ciclo de vida de las actividades apropiadamente.
- Los “widgets” son vistas que proporcionan elementos visuales e interactivos como un botón, un campo de texto, una casilla de verificación ...
- La forma más común de definir un diseño mediante el uso de vistas es con un archivo de diseño XML, separado de la programación. Excepto si utilizas el kit de herramientas JetPack Compose que no utiliza ficheros xml. Es un sistema de creación de interfaces declarativas para Android.

CONCEPTOS

AVD (Android Virtual Device o Dispositivo Virtual de Android)

- Un AVD es una instancia de emulador que permite modelar un dispositivo. Todo AVD contiene un perfil hardware, una imagen de sistema y el almacenamiento emulado.
- Se pueden crear tantos AVD como se quieran para probar las aplicaciones con diferentes configuraciones y son imprescindibles para comprobar el correcto funcionamiento de la aplicación al ejecutarse en diferentes dispositivos.



DESARROLLO

- **Android Studio Layout Editor.**

Es el editor de diseño que permite incorporar elementos en la interfaz sin necesidad de escribir líneas de código ni modificar los xml. Lo que encontrarás en el Layout Editor será:

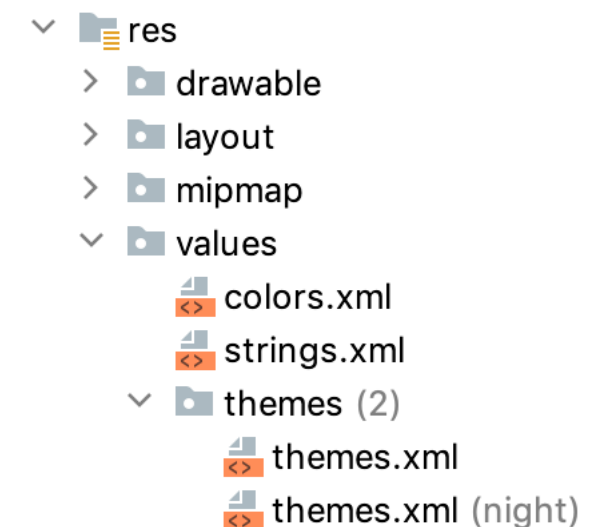
- A. **Palette:** lista de vistas que puedes incluir.
- B. **Component Tree:** jerarquía de vistas de tu diseño.
- C. **Barra de herramientas:** botones para configurar la apariencia de tu diseño.
- D. **Ajuste de diseño:** diseño en la vista Design o Blueprint, o en ambas.
- E. **Attributes:** controles para los atributos de la vista seleccionada y que te permitirá modificar las propiedades.

DESARROLLO

● Recursos de Android.

En Android, Recurso es un concepto amplio y hace referencia a muchos elementos desde imagenes, textos, estilos, etc... que la aplicación necesita para funcionar. Los recursos se pueden agrupar en ficheros de recursos y éstos se ubican en la carpeta /res organizados por categorías y que le va a permitir funcionar con diferentes configuraciones. Como por ejemplo los ficheros con los diseños (layout), o los contenidos en la carpeta res/values donde encontramos el fichero **strings.xml** encargado de contener los textos de la aplicación y parametrizarlos .

Strings.xml es un archivo de recursos de strings en el que deberías especificar todas las strings. Esto te permite controlarlas en una única ubicación y externo al código, lo que permite encontrarlas, actualizarlas y localizarlas más fácilmente. Y a esto le llamamos Proveedor de Recursos.



DESARROLLO

Provisión de recursos.

- Debes externalizar los recursos para aplicaciones como imágenes y strings de tu código, para que puedas mantenerlos de forma independiente. También debes proporcionar recursos alternativos para diferentes configuraciones de dispositivos. En tiempo de ejecución, Android utilizará el recurso adecuado según la configuración detectada.
- Ejemplo, puedes proporcionar un diseño de interfaz de usuario diferente según el tamaño de la pantalla, o strings diferentes según la configuración de idioma.
- Una vez que externalizas los recursos para tu aplicación, puedes acceder a ellos mediante los ID de recursos que se generan en la clase R de tu proyecto.

DESARROLLO

Directorios de recursos admitidos dentro del directorio res/

/animator	Archivos XML que definen animaciones de propiedades
/anim	Archivos XML que definen animaciones de interpolación de movimientos
/color	Archivos XML que definen una lista de estados de colores.
/drawable	Archivos de mapas de bits
/mipmap	Archivos de elemento de diseño para diferentes densidades de los íconos lanzadores.
/layout	Archivos XML que definen el diseño de una interfaz de usuario.
/menu	Archivos XML que definen menús de aplicaciones, como un menú de opciones, un menú contextual o un submenú.
/raw	Archivos arbitrarios para guardar sin procesar.
/values	Archivos XML que contienen valores simples, como strings, enteros y colores.
/xml	Archivos XML arbitrarios que se pueden leer en tiempo de ejecución.

DESARROLLO

Diseños flexibles.

- Independientemente de las dependencias hardware, debes crear diseños flexibles, incluidos los cambios de tamaño de pantalla. Para ello tienes el `ConstraintLayout`, que permite especificar la posición y el tamaño de cada vista en función a otras vistas.
- Debes evitar los tamaños fijos y referenciarlos con:
 - `"wrap_content"`: debe establecer el tamaño necesario para adaptarse al contenido de esa vista.
 - `"match_parent"`: hace que la vista se expanda tanto como sea posible dentro de la vista superior (si se usa `ConstraintLayout`, debería ser `0dp`).
- Puedes crear diseños específicos para cada pantalla (ejemplo, un móvil y una Tablet) creando directorios `res/layout/` adicionales, por ejemplo, utilizando el calificador de ancho mínimo:

```
res/layout/main_activity.xml      # For handsets (smaller than 600dp available width)
res/layout-sw600dp/main_activity.xml  # For 7" tablets (600dp wide and bigger)
```


DESARROLLO

- El fichero R.java es generado por el compilador y se trata de una clase llena de variables estáticas en las que se identifica cada tipo de recurso. Como hemos visto, Android hace uso de ficheros xml para definir las estructuras, que van a ser leídas cuando la aplicación arranque. Ahora bien, estamos desarrollando para dispositivos móviles, por lo que trabajar con XML directamente puede hacer el funcionamiento muy lento. Por lo tanto Android después de leer el fichero XML carga todas las estructuras que se han solicitado en memoria y mantiene el fichero R como referencia directa a los recursos que se han cargado y así el acceso será directo.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

DESARROLLO

Intent.

- Es un objeto de acción que se utiliza para solicitar una acción de otro componente. Por ejemplo, se utiliza principalmente para:
 - Iniciar una actividad -> `startActivity()`
 - Iniciar un servicio -> `startService()`
 - Entregar un mensaje -> `sendBroadcast()`
- Para comenzar la actividad se pasa la Intent a `startActivity` (la intent tiene la actividad a iniciar). Si se espera un resultado al finalizar la actividad se llama a `startActivityForResult()`.
- Las **intents** pueden ser:
 - **Explícitas:** especifican qué componente se debe iniciar mediante su nombre.
 - **Implícitas:** no se nombra el componente pero se indica la acción general que debe realizarse para que otra aplicación la maneje.

DESARROLLO

- Para las intents implícitas es para lo que se dispone de un filtro de intents, que será una expresión en el archivo de manifiesto que indica el tipo de intents que el componente podría recibir. Si no declara ningún filtro de intent para una actividad, solo se la puede iniciar con una intent explícita.

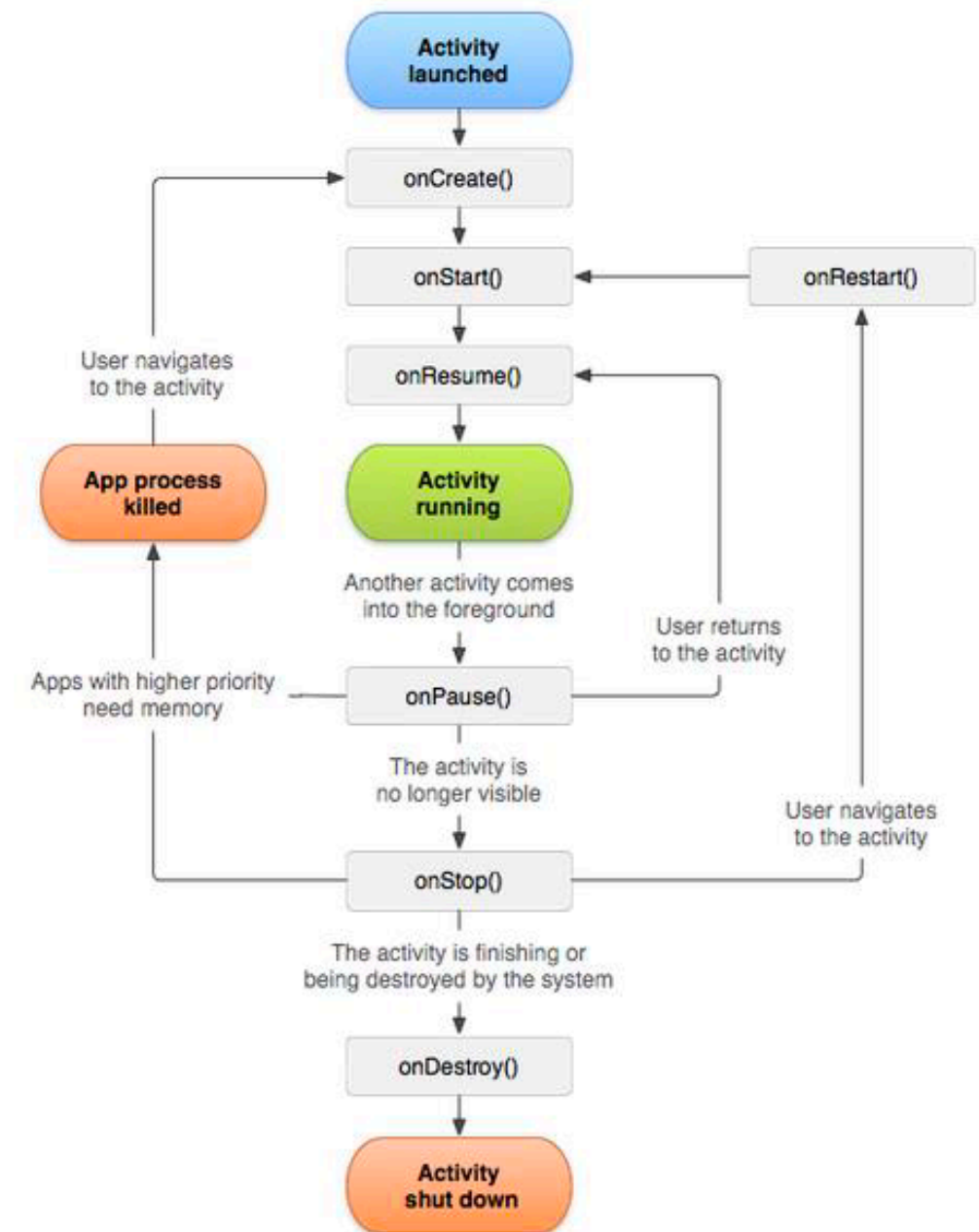
```
<intent-filter android:icon="drawable resource"
              android:label="string resource"
              android:priority="integer" >
    . . .
</intent-filter>
```

Para garantizar que una aplicación sea **segura**, siempre debe usarse una *intent* explícita cuando inicie un SERVICIO y no deben declararse *filtros de intents* para los servicios.

DESARROLLO

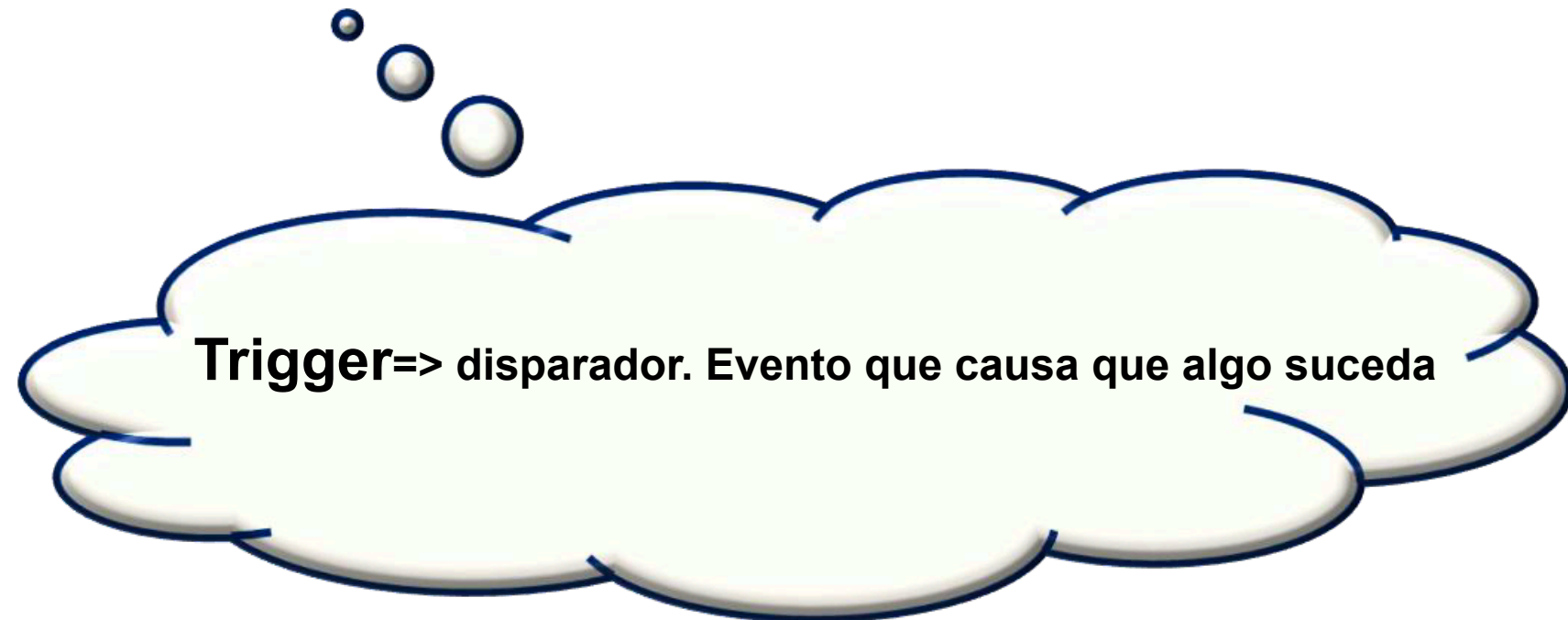
Ciclo de vida de una actividad.

- **onCreate()**: la implementación debe inicializar los componentes básicos de la aplicación. Se debe hacer la llamada a setContentView().
- **onStart()**: la actividad se hace visible al usuario.
- **onResume()**: en este punto comienza a interactuar con el usuario y recoge las entradas del usuario.
- **onPause()**: la actividad pierde el foco, aunque esté visible.
- **onStop()**: ya no será visible al usuario.
- **onRestart()**: restaura el estado de la actividad al momento que paró.
- **onDestroy()**: los recursos son liberados y finaliza efectivamente la aplicación.



DESARROLLO.

- Los cambios de estado de una actividad se deben a diferentes eventos: triggers del usuario o triggers del sistema.



DESARROLLO

- **Widgets.** Componentes visuales para entrada de información.
Algunos controles típicos y la clase asociada.

Texto

- TextView
- EditText
- AutoCompleteText
- View

Botón

- Button

Casilla de verificación

- CheckBox

Botón de opción

- RadioButton
- RaidoGroup

Switches

- ToggleButton

ComboBox o listas desplegadas

- Spinner

Imágenes

- ImageView
- ImageButton

DESARROLLO

- Muchas de las propiedades/atributos son comunes: TextColor, TextSize...
- Para los componentes que responden a eventos HAY que implementar el Listener (por ejemplo OnClickListener...)
- Las acciones a realizar son:
 - Definir/diseñar el componente (de forma gráfica o en XML)
 - Asignar propiedades
 - Dependiendo de si requiere listener para actuar según opción del usuario.
 - Códificar el listener....

DESARROLLO.

- **Fragmentos.**

- Un FRAGMENT representa un comportamiento o una parte de la interfaz de usuario en una activity. Puedes combinar múltiples fragmentos en una sola actividad para crear una IU multipanel y volver a usar un fragmento en múltiples actividades.
- Su principal finalidad es la de ampliar parte de la lógica utilizada para la navegación entre pantallas o Activities, pudiendo definir varios Fragments dentro de una misma Activity, interaccionando entre ellos.
- Un Fragment debe estar embebido dentro de una Activity, por lo que el ciclo de vida de un Fragment está ligado al de la Activity dónde se ha definido:
 - A. Al pausar una Activity se pausarán todos los Fragments definidos dentro de esta.
 - B. Si se destruye una Activity, todos los Fragments que contenga serán también destruidos.
 - C. Si la Activity se encuentra en ejecución, es posible manipular de manera independiente cada Fragment, inclusive destruirlo.

DESARROLLO

- **Permisos.**
- Los permisos tienen como objetivo proteger la privacidad de los usuarios. Todas las apps Android deben manejar y solicitar permiso para acceder a los datos sensibles como los contactos, o bien, propias del sistema como el uso de la cámara. Por defecto, las aplicaciones no cuentan con estos permisos. Es decir, trabajan en una zona con acceso limitado y si necesitan acceder a recursos o información externos, debe indicarlo.
- La etiqueta `<uses-permission>` en el Manifest, incluye las líneas para lo que se requiere permiso.
- Suponemos que la app necesita acceso a Internet =>
- ejemplo:
- `<uses-permission android:name="android.permission.INTERNET"/>` `<uses-permission android:name="android.permission.SEND_SMS"/>`
- etc...